

Senior Engineer Take-Home

Real-Time Market Data Hub with MCP Interface

Issued: April 20, 2026 | Duration: 2 weeks

Overview

Build a real-time crypto market data hub that ingests from Coinbase and distributes to multiple consumers through a subscribable pub/sub interface. The hub must be natively AI-interfaceable via an MCP (Model Context Protocol) server, so internal tools and LLM agents consume the same data through one unified surface. Scope is a single venue (Coinbase), but the architecture should make adding venues a simple exercise. We want one centralized hub that owns the upstream relationship, and manages the distribution to the consumers, and exposes itself well enough that an LLM given only the context documentation can use it correctly on first attempt.

The Core Shape

The system is designed around a single upstream stream, and many downstream connections. The hub acts as an intermediary between Coinbase and every internal consumer. Upstream, it owns a small number of WebSocket connections to Coinbase and is responsible for keeping them healthy. Downstream, any number of consumers connect, subscribe to topics, and receive the updates in real time. A centralised registry tracks both sides and reference-counts subscriptions: the hub only maintains an active Coinbase subscription while at least one consumer wants it.

Functional Requirements

1. Coinbase Ingestion

- Establish and maintain WebSocket connections to Coinbase WebSocket feed.
- Implement connection lifecycle management
- Consider Coinbase subscription constraints and message limits
- Provide clear concise documentation for all subscribed channels

2. Subscribable Topics

- Establish a clear and consistent naming convention: e.g. BTC-USD, ETH-USD, SOL-USD.
- Support multiple consumers per topic.
- Define and document your backpressure strategy for slow or overloaded consumers.
- Allow consumers to dynamically subscribe and unsubscribe from topics

3. Connection Registry

- Track every upstream (Coinbase) and downstream (consumer) connection in a centralised registry.
- Reference-count topic subscriptions so upstream subscriptions reflect actual downstream demand.
- Ensure tear down cleanly on disconnect preventing leaked subscriptions.
- Expose a status surface for active connections, uptime per connection, message rates.

4. MCP Server (Important)

Wrap the hub in an MCP server so AI agents can use it as a first-class tool. The MCP layer should expose, at minimum, tools to: list available topics, describe a topic's schema and example payload, subscribe to a topic and stream messages, and query current snapshot state (top of book, last trade, best bid/ask). Tool names, descriptions, and argument schemas must be written for an LLM consumer: action-verb names, strongly typed arguments, short descriptions that explain when to use the tool, not just what it does.

5. Documents That Teach AI Context (Important)

Documentation is a first-class deliverable, not an afterthought. Create a context document so that an LLM loaded with them can immediately and correctly drive the MCP tools. That means:

- Purpose, scope, and non-goals stated up front in plain language.
- Every topic's name, schema, update cadence, and a real example payload.
- Worked examples: 'Agent wants the current mid for BTC-USD: it should call X with Y and expect Z.'
- Failure modes spelled out: what does the agent see when a topic is stale, a connection drops, or a symbol is unknown.

Non-Functional Requirements

- Pick a language of your choice for fast development and iteration (Python, TypeScript)
- Production-shaped code: tests, structured logs, configuration files, Dockerfile, README.
- Package the system in a single-container or single-binary deployable.
- No database required. In-memory state is fine; but be explicit about what is lost on restart.

Deliverables

- Source repo (GitHub link preferred, zip acceptable).
- MCP server with clear setup instructions (this can be deployed locally)
- Context documentation markdown files in /docs.
- One-page architecture write-up covering design choices and trade-offs.
- 10-minute walkthrough of the system and your reasoning.

Evaluation Criteria

- Architecture judgement: strong engineering design with clear separation between ingestion, registry, pub/sub, and MCP layers with attention to scalability, maintainability and reliability. The system should be structured to support future AI usage ensuring both engineers and LLM-based agents can build and operate effectively.
- Correctness under stress: reconnections and stale states.
- LLM usability: how far a fresh agent gets when handed only your documents and MCP tool list.
- Code quality and ergonomics: readability, maintainability and how easily for another engineer to understand and extend the system.

Ground Rules

- Use of AI assistants are expected and encouraged be transparent on how you leveraged them.
- Ask clarifying questions early to resolve any ambiguity.